

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"**

РОЗРОБЛЕННЯ МОБІЛЬНОГО ЗАСТОСУНКУ

МЕТОДИЧНІ ВКАЗІВКИ

до лабораторної роботи №2
з дисципліни “Програмування для мобільних платформ”
спеціальності 121 ”Інженерія програмного забезпечення”

*Затверджено
на засіданні кафедри
програмного забезпечення.
Протокол № від 28.03.2025*

Львів 2025

Розроблення мобільного застосунку: Методичні вказівки до лабораторної роботи для студентів спеціальності “Інженерія програмного забезпечення” / Укл. Роман Кутельмах – Львів: Видавництво Національного університету “Львівська політехніка”, 2025. – 24 с.

Укладач: Кутельмах Р.К., к.т.н., старший викладач

Відповідальний за випуск: Федасюк Д.В. – д.т.н., проф., зав. кафедри

Рецензенти: Коротєєва Т. О. – к.т.н., доц., доц.

Тушницький Р. Б. – к.т.н., доц., доц.

МЕТА ВИКОНАННЯ ЛАБОРАТОРНОЇ РОБОТИ

Метою виконання лабораторної роботи є розроблення мобільного застосунку на обраній платформі.

Рекомендовані програмні засоби: Виконання роботи передбачається у середовищі Android Studio.

ТЕОРЕТИЧНА ЧАСТИНА

Мова програмування Dart

Створимо основну функцію-хаб для інших функцій:

```
void functionPlayground() {  
    classicalFunctions();  
    optionalParameters();  
}
```

Реалізація простих функцій на мові Dart виглядає наступним чином:

```
void printMyName(String name) {  
    print('Hello $name');  
}  
  
int add(int a, int b) {  
    return a + b;  
}  
  
int factorial(int number) {  
    if (number <= 0) {  
        return 1;  
    }  
  
    return number * factorial(number - 1);  
}  
  
void classicalFunctions() {  
    printMyName('Anna');  
    printMyName('Michael');  
  
    final sum = add(5, 3);  
    print(sum);  
  
    print('10 Factorial is ${factorial(10)}');  
}
```

У Dart можна використовувати *необов'язкові позиційні аргументи*. Якщо помістити перелік аргументів функції в квадратні дужки, то їх можна пропустити без помилок компіляції:

```
void unnamed([String? name, int? age]) {  
    final actualName = name ?? 'Unknown';  
    final actualAge = age ?? 0;  
    print('$actualName is $actualAge years old.');
```

Є також підтримка *іменованих необов'язкових аргументів* (задаються фігурними дужками):

```
void named({String? greeting, String? name}) {  
    final actualGreeting = greeting ?? 'Hello';  
    final actualName = name ?? 'Mystery Person';  
    print('$actualGreeting, $actualName!');
```

Іменовані аргументи допомагають уникнути двозначності. Необов'язкові та необов'язкові іменовані аргументи також підтримують значення по замовчуванню. Якщо параметр пропущений під час виклику функції, замість null буде використано значення по замовчуванню. В описі функції спочатку можна розмістити обов'язкові, а потім необов'язкові аргументи:

```
String duplicate(String name, {int times = 1}) {  
    final merged = StringBuffer(name);  
    for (var i = 1; i < times; i++) {  
        merged.write(' $name');
```

А зараз, щоб побачити в дії всі ці елементи, реалізуємо наступну функцію:

```

void optionalParameters() {
  unnamed('Huxley', 3);
  unnamed();

  // Notice how named parameters can be in any order
  named(greeting: 'Greetings and Salutations');
  named(name: 'Sonia');
  named(name: 'Alex', greeting: 'Bonjour');

  final multiply = duplicate('Mikey', times: 3);
  print(multiply);
}

```

Тепер оновимо метод **main**, щоб можна було виконувати ці функції:

```

main() {
  functionPlayground();
}

```

У Dart дуже поширеним є використання замикань. Щоб додати замикання до функції, ви повинні, по суті, визначити іншу сигнатуру функції всередині цієї функції:

```

void callbackExample(void Function(String value) callback) {
  callback('Hello Callback');
}

```

В методі **callbackExample** створимо ще одну просту функцію, яка виводить свій параметр:

```

void printSomething(String value) {
  print(value);
}

```

Використання замикань може виявитися досить складним. Щоб спростити це, Dart використовує ключове слово **typedef**. У методі **main** створимо **typedef** з ім'ям **NumberGetter**, який буде функцією, що повертає ціле число:

```

typedef NumberGetter = int Function();

```

Наступна функція прийме **NumberGetter** в якості параметра і викличе його:

```
int powerOfTwo(NumberGetter getter) {
    return getter() * getter();
}
```

Об'єднаємо це разом із функцією, яка використовуватиме всі приклади замикання:

```
void consumeClosure() {
    int getFour() => 4;
    final squared = powerOfTwo(getFour);
    print(squared);

    callbackExample(printSomething);
}
```

По суті, замикання — це функція, яка зберігається в змінній, котру можна викликати пізніше. Замикання часто використовуються для зворотних викликів, наприклад, коли користувач натискає кнопку або коли програма отримує дані від виклику з мережі. Ми вже ознайомились з двома способами визначення замикання: 1) прототипи функцій та 2) typedefs. Ми викликаємо функцію та надаємо наше замикання, яке повертає число 4. Цей код також використовує синтаксис стрілок, який дозволяє вам писати будь-яку функцію, що займає один рядок без дужок. Для однорядкових функцій замість дужок можна використовувати синтаксис стрілок, =>.

Розглянемо наступний приклад:

```
int dayOfWeek = 7;
//String myDay = getDay(dayOfWeek);

var myDay = switch (dayOfWeek) {
    1 => 'Monday',
    2 => 'Tuesday',
    3 => 'Wednesday',
    4 => 'Thursday',
    5 => 'Friday',
    6 => 'Saturday',
    7 => 'Sunday' ,
    _ => 'Invalid day' //Default value
};
print(myDay);
```

У цьому випадку ви можете опустити ключове слово case, а в тілі case вказати один вираз замість ряду операторів. Ключові слова case розділяються комою, а в case по замовчуванню слід використовувати підкреслення (_). Це

набагато компактніший спосіб написати оператор switch, якщо потрібен один вираз.

Одна з головних змін, яка була введена в Dart 3 – це **records (записи)**. Це типи, які містять дані користувача. Приклад:

```
var person = (name: 'Clark', age: 42);  
  
print (person.name);
```

В цьому випадку person – це **record expression (вираз запису)**: незмінний набір полів (іменованих, як у цьому прикладі, або позиційних), розділених комою. Це дуже ефективний спосіб поєднання різних значень в один тип. При оголошенні змінної також можна використовувати **record type annotations (анотації типу запису)**:

```
({String name, int age}) person = (name: 'Clark', age: 42);  
print (person.name);
```

В цьому випадку **person** було явно оголошено з його типом. Тепер розглянемо інший приклад:

```
void main() {  
  var (String name, int age) = getPerson({'name': 'Clark', 'age': 42});  
  print('$name is $age years old.');
```

```
(String, int) getPerson(Map<String, dynamic> json) {  
  return (json['name'] as String, json['age'] as int);  
}
```

getPerson – метод, який повертає запис типу **String, int**, але замість повернення **Map** або повного об'єкта, він повертає запис із значеннями в дужках.

Наведена нижче інструкція:

```
var (String name, int age) = getPerson({'name': 'Clark', 'age': 42});
```

називається **патерн**. Він перетворює запис на дві нові змінні: ім'я та вік.

Класи в Dart не дуже відрізняються від тих, які можна знайти в інших мовах об'єктно-орієнтованого програмування (ООП).

Спочатку визначимо клас **Name**, який є об'єктом, що зберігає ім'я та прізвище людини:

```
class Name {  
    final String first;  
    final String last;  
    Name(this.first, this.last);  
  
    @Override  
    String toString() {  
        return '$first $last';  
    }  
}
```

Визначимо підклас **OfficialName**. Процедура та ж сама, що й для класу **Name**, але він також матиме офіційний титул:

```
class OfficialName extends Name {  
    // Private properties begin with an underscore  
    final String _title;  
  
    // You can add colons after constructor  
    // to parse data or delegate to super  
    OfficialName(this._title, String first, String last)  
        : super(first, last);  
  
    @Override  
    String toString() {  
        return '$_title. ${super.toString()}';  
    }  
}
```

Тепер ми можемо побачити всі ці концепції в дії за допомогою методу **classPlayground**:

```
void classPlayground() {  
    final name = OfficialName('Mr', 'Clark', 'Kent');  
    final message = name.toString();  
    print(message);  
}
```


Відмінність Dart від інших мов ООП полягає у відсутності явних ключових слів для інтерфейсів та абстрактних класів. Існує три ключові слова для побудови зв'язків між класами:

extends (розширення) – це ключове слово використовується з будь-яким класом, де ви хочете розширити функціональність суперкласу. Клас може розширюватись лише одним класом. Dart не підтримує множинне наслідування.

implements (реалізація) – використовувати інструменти, коли хочете створити власну реалізацію іншого класу, оскільки **всі класи є неявними інтерфейсами**. Якщо клас **FullName** реалізує клас **Name**, усі функції, визначені в класі **Name**, повинні бути реалізовані. Це означає, що коли ви реалізуєте клас, **ви не наслідуєте сам код**, лише його визначення. Класи можуть реалізовувати будь-яку кількість інтерфейсів.

with (поєднання) - використання **міксину**. У Dart клас може лише розширювати інший клас. **Міксини** дозволяють повторно використовувати код класу в кількох ієрархіях класів. Це означає, що міксини дозволяють отримувати блоки коду без необхідності створювати підкласи.

Якщо ви хочете моделювати групи схожих даних, вашим рішенням буде створення колекцій (**collections**). Колекція містить групу елементів. У Dart існує безліч типів колекцій, але ми зосередимося на трьох найпопулярніших: List, Map та Set.

Ці три основні типи колекцій можна знайти в кожній мові програмування, але іноді під іншою назвою:

Dart	Java	Swift	JavaScript
List	ArrayList	Array	Array
Map	HashMap	Dictionary	Object
Set	HashSet	Set	Set

Наступна функція демонструє оголошення, додавання/вилучення зі списку:\

```
void listPlayground() {  
    // Creating with list literal syntax  
    final List<int> numbers = [1, 2, 3, 5, 7];  
    numbers.add(11);  
    numbers.addAll([8, 17, 35]);  
    // Assigning via subscript  
    numbers[1] = 15;  
    print('The second number is ${numbers[1]}');  
    for (int number in numbers) {  
        print(number);  
    }  
}
```

Приклад роботи з множиною (Set):

```
void setPlayground() {  
    // Set literal, similar to Map, but no keys  
    final Set<String> ministers = {'Justin', 'Stephen', 'Paul', 'Jean',  
    'Kim', 'Brian'};  
    ministers.addAll({'John', 'Pierre', 'Joe', 'Pierre'}); //Pierre is  
    a duplicate, which is not allowed in a set.  
  
    final isJustinAMinister = ministers.contains('Justin');  
    print(isJustinAMinister);  
  
    // 'Pierre' will only be printed once  
    // Duplicates are automatically rejected  
    for (String primeMinister in ministers) {  
        print('$primeMinister is a Prime Minister.');    }  
}
```

Ще одна функція Dart – це можливість включати оператори управління потоком безпосередньо у колекціях. Ця функція є одним із небагатьох прикладів, коли Flutter безпосередньо впливає на мову програмування. Ви можете включити оператори **if**, цикли **for** і **spread**-оператори безпосередньо в ваших викликах колекції.

```
void collectionControlFlow() {  
    final addMore = true;  
    final randomNumbers = [  
        34,  
        232,  
        54,  
        32,  
        if (addMore) ...[  
            123,  
            258,  
            512,  
        ],  
    ];  
  
    final doubled = [  
        for (int number in randomNumbers) number * 2,  
    ];  
  
    print(doubled);}
```

В Dart **higher-order functions** (функції вищого порядку) є функціями, які приймають одну чи декілька функцій як аргументи або повертають функцію. Вони є потужним інструментом для керування даними, оскільки дозволяють писати багаторазовий і модульний код, який можна легко адаптувати до різних сценаріїв. Функції вищого порядку є одним із основних інструментів функціонального програмування. Створимо глобальну змінну під назвою **data**:

```
List<Map> data = [
  {'first': 'Nada', 'last': 'Mueller', 'age': 10},
  {'first': 'Kurt', 'last': 'Gibbons', 'age': 9},
  {'first': 'Natalya', 'last': 'Compton', 'age': 15},
  {'first': 'Kaycee', 'last': 'Grant', 'age': 20},
  {'first': 'Kody', 'last': 'Ali', 'age': 17},
  {'first': 'Rhodri', 'last': 'Marshall', 'age': 30},
  {'first': 'Kali', 'last': 'Fleming', 'age': 9},
  {'first': 'Steve', 'last': 'Goulding', 'age': 32},
  {'first': 'Ivie', 'last': 'Haworth', 'age': 14},
  {'first': 'Anisha', 'last': 'Bourne', 'age': 40},
  {'first': 'Dominique', 'last': 'Madden', 'age': 31},
  {'first': 'Kornelia', 'last': 'Bass', 'age': 20},
  {'first': 'Saad', 'last': 'Feeney', 'age': 2},
  {'first': 'Eric', 'last': 'Lindsey', 'age': 51},
  {'first': 'Anushka', 'last': 'Harding', 'age': 23},
  {'first': 'Samiya', 'last': 'Allen', 'age': 18},
  {'first': 'Rabia', 'last': 'Merrill', 'age': 6},
  {'first': 'Safwan', 'last': 'Schaefer', 'age': 41},
  {'first': 'Celeste', 'last': 'Aldred', 'age': 34},
  {'first': 'Taio', 'last': 'Mathews', 'age': 17},
];
```

Першою функцією вищого порядку є **map**. Її мета — отримати дані в одному форматі та швидко перетворити їх в інший формат. У наступному прикладі ми будемо використовувати функцію **map**, щоб перетворити рядок пар з ключовими значеннями **Map** на перелік об'єктів **Name** з чітко визначеним типом:

```
List<Name> mapping() {
    // Transform the data from raw maps to a strongly typed model
    final names = data.map<Name>((Map rawName) {
        final first = rawName['first'];
        final last = rawName['last'];
        return Name(first, last);
    }).toList(); //don't forget the toList() method!

    return names;
}
```

Тепер, коли дані чітко розподілені за типами, ми можемо скористатися відомою схемою для сортування переліку імен. Щоб використати функцію сортування для упорядкування імен за алфавітом з використанням лише одного рядка коду, додайте зазначену нижче функцію:

```
void sorting() {
    final names = mapping();

    // Sort the list by last name

    names.sort((a, b) => a.last.compareTo(b.last));

    print('');
    print('Alphabetical List of Names');
    names.forEach(print);
}
```

Часто бувають сценарії, коли вам потрібно отримати підмножину даних. За допомогою наведеної нижче функції вищого порядку можна одержати новий перелік імен, які починаються лише з літери *M*:

```

void filtering() {
    final names = mapping();
    final onlyMs = names.where((name) => name.last.startsWith('M'));

    print('');
    print('Filters name list by M');
    onlyMs.forEach(print);
}

```

Скорочення переліку — це дія отримання одного значення з усієї колекції. У наступному прикладі ми виконаємо скорочення для обчислення середнього віку усіх людей зі списку:

```

void reducing() {
    // Merge an element of the data together
    final allAges = data.map<int>((person) => person['age']);
    final total = allAges.reduce((total, age) => total + age);
    final average = total / allAges.length;

    print('The average age is $average');
}

```

Останній приклад вирішує проблему, з якою ви можете зіткнутися, коли є колекції, що містять інші колекції, і вам потрібно видалити частину такого вмісту. Ця функція демонструє, як ми можемо зробити один лінійний перелік з двовимірної матриці:

```

void flattening() {
    final matrix = [
        [1, 0, 0],
        [0, 0, -1],
        [0, 1, 0],
    ];

    final linear = matrix.expand<int>((row) => row);
    print(linear);
}

```

Для більшої переконливості ми могли б ще більше скоротити приклад коду та додати більше функцій:

```
final names = data
    .map<Name>((row) => Name(row['first'], row['last']))
    .where((name) => name.last.startsWith('M'))
    .where((name) => name.first.length > 5)
    .toList(growable: false);
```

До цього моменту ми бачили, як Dart слідує багатьом зразкам інших сучасних мов. Dart, у певному сенсі, бере найкраще з багатьох мов — тут є й виразність JavaScript і безпечність типів Java. Однак є деякі особливості, які є унікальними для Dart. Однією з таких особливостей є оператор **cascade** (**..**). Ось як можна реалізувати приклад каскадного оператора:

```
class UrlBuilder {
    String scheme = '';
    String host = '';
    List<String> routes = [];

    @override
    String toString() {
        final paths = [host, if (routes != []) ...routes];
        final path = paths.join('/');
        return '$scheme://$path';
    }
}
```

```
void cascadePlayground() {
  final url = UriBuilder()
    ..scheme = 'https'
    ..host = 'dart.dev'
    ..routes = [
      'guides',
      'language',
      'language-tour#cascade-notation',
    ];

  print(url);
}
```

Каскадний оператор не використовується виключно для шаблону конструктора. Він також, до речі, може використовуватися для каскадування схожих операцій на тому самому об'єкті. Додайте наступний код всередину функції **cascadePlayground**:

```
final numbers = [42, 88, 53, 232, 55]
  ..insert(0, 8)
  ..sort((a, b) => a.compareTo(b));

print('The largest number in the list is ${numbers.last}');
```

За допомогою каскадного оператора можна об'єднати непов'язані оператори в простий плавний ланцюжок викликів функцій.

Програмування на Flutter. Віджети

У Flutter доступно два типи візуальних віджетів: віджети без стану та віджети зі станом. Віджет **без стану** простий, легкий і продуктивний. Flutter може відтворювати сотні віджетів без стану. Почніть роботу зі створення нового проєкту Flutter під назвою **flutter_layout**, за допомогою IDE або командного рядка.

Відкрийте **main.dart** і видаліть усе. Потім введіть наступний код у редакторі:

```

void main() => runApp(const StaticApp());

class StaticApp extends StatelessWidget {
  const StaticApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const ImmutableWidget(),
    );
  }
}

```

В папці **lib** проєкту створимо новий файл - **immutable_widget.dart**. Ви побачите порожній файл. Для створення віджетів без стану існують шаблони, але їх можна створити й за допомогою простого фрагменту коду. Просто введіть **stless**. При натисканні кнопки *Enter* на екрані з'явиться шаблон (рис. 1).

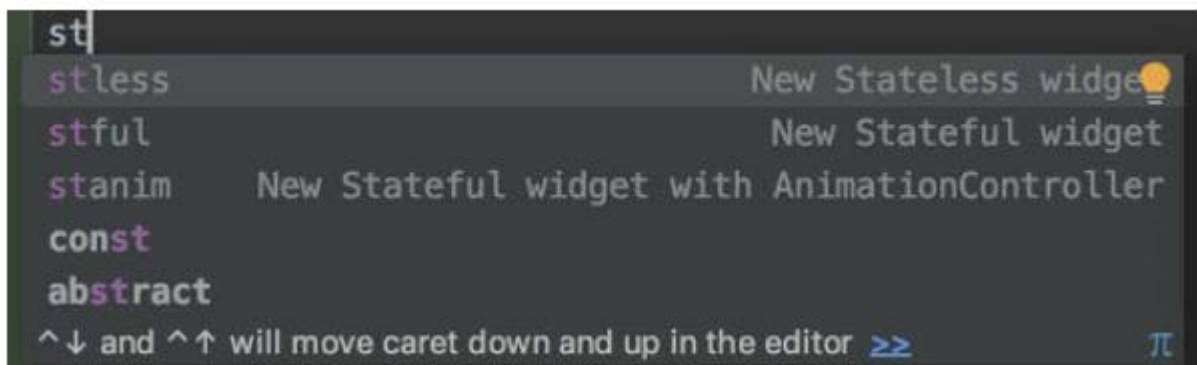


Рис. 1. Шаблон створення нового класу віджета

Для створення нового віджету без стану, наберемо наступний код:

```
import 'package:flutter/material.dart';

class ImmutableWidget extends StatelessWidget {
  const ImmutableWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.green,
      child: Padding(
        padding: const EdgeInsets.all(40),
        child: Container(
          color: Colors.purple,
          child: Padding(
            padding: const EdgeInsets.all(50.0),
            child: Container(
              color: Colors.blue,
            ),
          ),
        ),
      ),
    );
  }
}
```

Віджет створено. Тепер можна повернутись до `main.dart` та імпортувати файл **`immutable_widget.dart`**. Для запуску додатку в Android Emulator, натисніть

кнопку запуску. На екрані має бути відображено три вкладені один в одного поля (рис. 2).

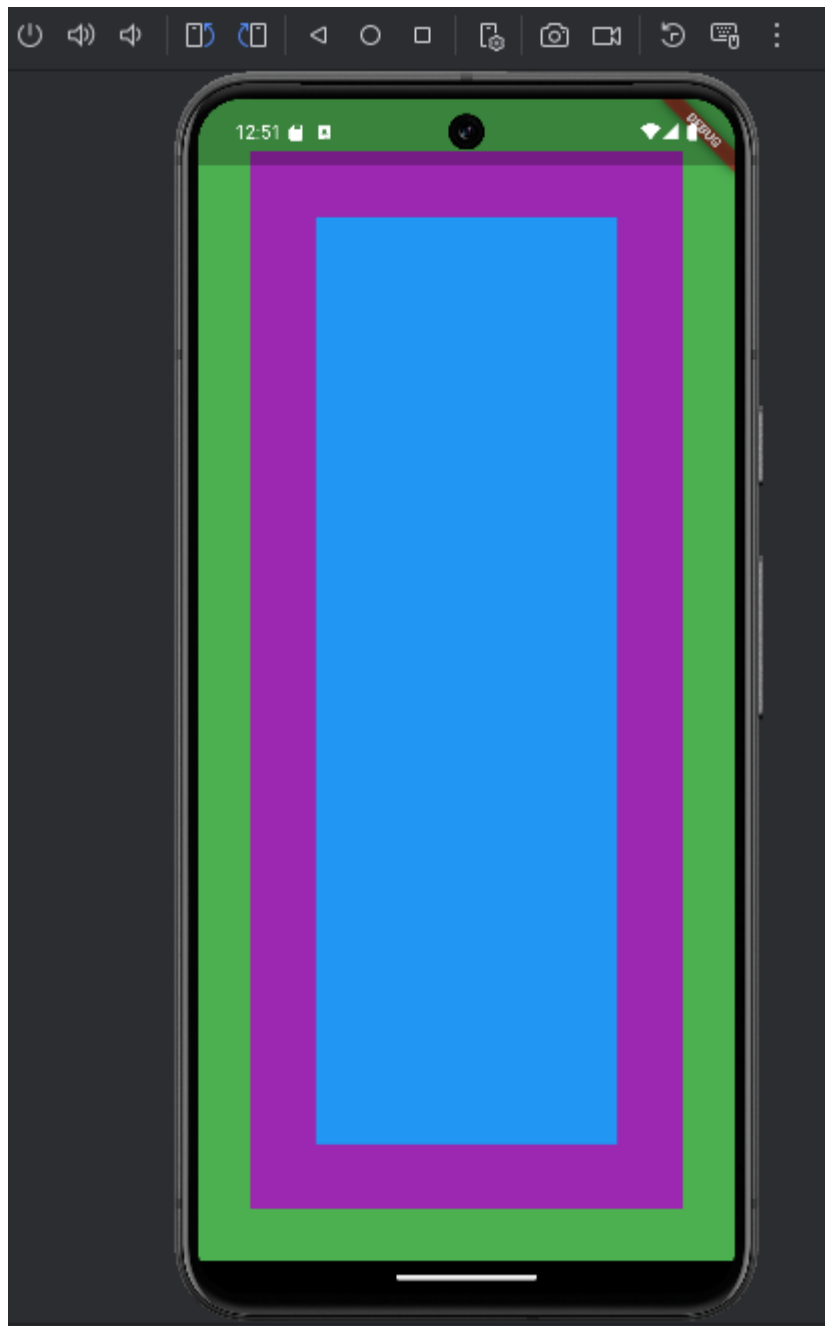


Рис. 2. Приклад роботи створеного віджета на емуляторі

Віджет **Scaffold** використовується, щоб додати **AppBar** в верхню частину екрану та випадаючий **drawer**, який можна витягнути зліва. Напевно, одним із найпопулярніших віджетів у додатку є **AppBar**. Це постійний заголовок, який розташований у верхній частині екрана та дозволяє переходити з одного екрана на інший. Додамо до **Scaffold** наступний код:

```

return Scaffold(
  appBar: AppBar(
    backgroundColor: Colors.indigo,
    title: const Text('Welcome to Flutter'),
    actions: const [
      Padding(
        padding: EdgeInsets.all(10.0),
        child: Icon(Icons.edit),
      ),
    ],
  ),
  body: Center(
    ...

```

Після центрування віджету та додавання **AppBar** вигляд екрану оновиться (рис. 3).

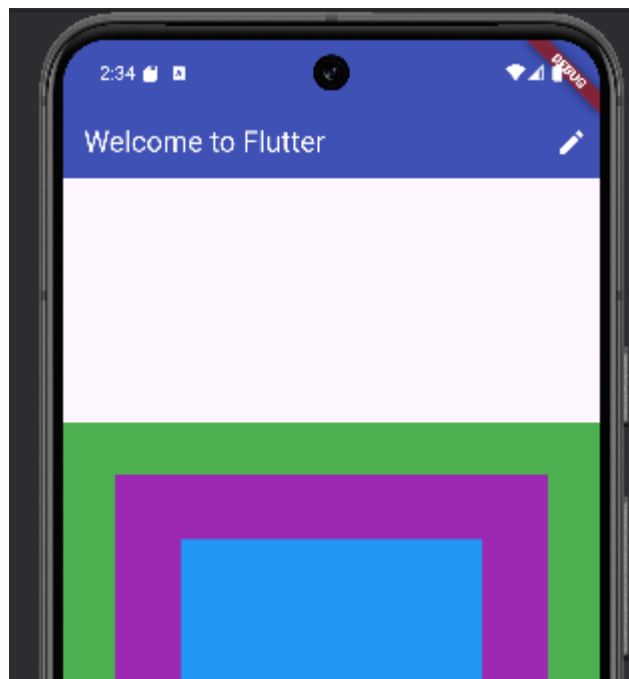


Рис. 3. Scaffold та AppBar у мобільному застосунку

Додамо до застосунку **drawer**. Одразу після **body** наберемо у **Scaffold** наступний код:

```
body: Center(
  child: AspectRatio(
    aspectRatio: 1.0,
    child: ImmutableWidget(),
  ),
),
drawer: Drawer(
  child: Container(
    color: Colors.lightBlue,
    child: const Center(
      child: Text("I'm a Drawer!"),
    ),
  ),
),
),
```

Фінальна версія застосунку з AppBar тепер матиме так звану іконку «гамбургер». При натисканні на неї на екрані з'являється панель з **drawer** (рис. 4).

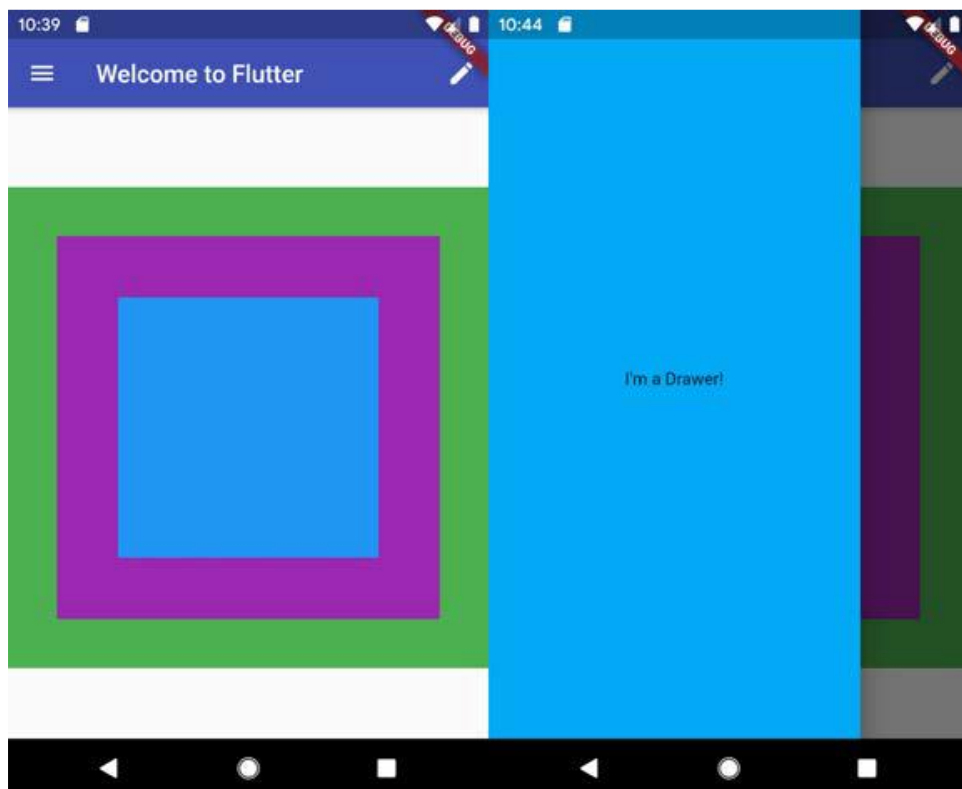


Рис. 4. Панель drawer у застосунку

ЛІТЕРАТУРА

1. Kutelmakh R., Development features of mobile apps in the context of user experience, Modern engineering and innovative technologies, 27-01, 2023, pp. 123-128
2. М.В. Давидов, А.Б. Демчук, О.В. Лозинська, Програмне забезпечення мобільних пристроїв: навчальний посібник – Львів: Видавництво «Новий Світ-2000» 2020. – 218 с.
3. Alessandria S., Flutter Cookbook: 100+ step-by-step recipes for building cross-platform, professional-grade apps with Flutter 3.10.x and Dart 3.x, 2nd Edition 2nd Edition, Packt Publishing, 2023, 712 pp.
4. Wangereka H., Mastering Kotlin for Android 14: Build powerful Android apps from scratch using Jetpack libraries and Jetpack Compose, Packt Publishing, 2024, 370 pp.

ЗАВДАННЯ ДЛЯ ЛАБОРАТОРНОЇ РОБОТИ

Розробити мобільний застосунок за допомогою сучасних інструментів та технологій вибраної мобільної платформи. У межах застосунку реалізувати функції, що відображають та змінюють динамічні дані, а також використовують чотири або більше з наведених нижче рекомендованих елементів:

- **База даних (БД), динамічні дані.** Використання бази даних для збереження і опрацювання інформації.
- **Аналітика.** Інтеграція збору статистики використання застосунку.
- **Машинне навчання.** Інтеграція класифікації даних на основі бібліотек ML.
- **Локалізація та доступність.** Застосування елементів доступності та підтримка іншої мови.
- **Світла і темна теми, відповідність офіційним рекомендаціям платформи.** Налаштування теми (light/dark mode) та дизайну згідно з офіційними порадами обраної платформи.
- **Анімації.** Використання анімацій для плавних переходів між екранами чи елементами інтерфейсу.
- **Нотифікації.** Реалізація сповіщень для інформування користувача про важливі події.
- **Веб-сервіс.** Забезпечення обміну даними з віддаленим сервером (через REST API, тощо).

- **Мультимедіа, сенсори.** Робота з камерою, мікрофоном, відтворення аудіо чи відео або використайте будь-які наявні сенсори пристрою (акселерометр, компас, гіроскоп тощо).
- **Ефективний інтерфейс користувача.** Організація інтерфейсу користувача із врахуванням принципів UX/UI: зручне керування, логічна структура та швидка взаємодія.

Критерії оцінювання роботи

1. **Знання платформи та якість коду.** Оцінюється відповідність коду кращим практикам, читабельність, оптимізація та відсутність критичних помилок.
2. **Самостійність виконання.**
3. **Використання чотирьох рекомендованих елементів.** Продемонструйте щонайменше чотири рекомендовані елементи з переліку.
4. **Складність завдання.** Оцінюється рівень технічної складності реалізованих рішень, використання інструментів і бібліотек.
5. **Архітектура.** Використання основних принципів архітектури (Model-View-ViewModel, Hilt або аналогічні засоби залежно від платформи та інструментарію).

Звіт повинен включати:

1. Короткі теоретичні відомості про обрану мобільну платформу.
2. Опис основних класів.
3. Основний код програми.
4. Опис архітектурних рішень.
5. Скріншоти працюючого застосунку на смартфоні або емуляторі.

КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке Null Safety і як воно реалізоване в Dart?
2. Як використовувати `async/await`?
3. Які існують способи обробки винятків (exceptions) у Dart?
4. Яка різниця між `StatelessWidget` та `StatefulWidget`?
5. Як оголосити власний `StatefulWidget`? Опишіть принцип взаємодії з об'єктом `State`.
6. У чому полягає призначення методу `build(BuildContext context)`, і чому важливо не тримати там складну логіку?

7. Як вбудовувати один виджет у інший і як відбувається процес відмалювання (rendering) елементів інтерфейсу?
8. Як ви розумієте Widget Tree?
9. Які є основні layout-віджети у Flutter (наприклад, Column, Row, Stack, Expanded, Flexible)?
10. Які є способи реалізувати анімації у Flutter (зокрема, використання AnimationController, AnimatedBuilder, AnimatedWidget, TweenAnimationBuilder та ін.)?
11. Як створити AnimationController? Що таке vsync і навіщо він потрібен?
12. Чим корисні Tween і Curve при налаштуванні плавності та параметрів анімації?
13. Яка різниця між hero animations та звичайними анімаціями?
14. Як у Flutter/Dart правильно обробляти довготривалі операції, щоб не блокувати основний UI-потік?
15. Яка різниця між Future.then(...), async/await і FutureBuilder?
16. Поясніть, навіщо існують Streams. Коли краще використовувати їх замість Future?
17. Як використовувати StreamBuilder у побудові UI?
18. Як реалізувати HTTP-запити в Flutter? Яку бібліотеку для цього найчастіше використовують?
19. Які основні можливості надає Firebase для Flutter-застосунків (Firestore, Authentication, Cloud Storage тощо)?
20. Як підключити свій Flutter-застосунок до Firebase (конфігурація, налаштування)?
21. Чим відрізняється робота з Cloud Firestore від Realtime Database?
22. Як реалізувати аутентифікацію у Firebase?
23. Як налаштувати Push Notifications через Firebase Cloud Messaging?
24. Як працюють Routing і Navigation у Flutter? Які є способи організувати навігацію між екранами?
25. Що таке BLoC?
26. Як ви розумієте pubspec.yaml? Яка його роль у проєкті?
27. Як правильно організувати структуру файлів і папок у Flutter-проєкті?

НАВЧАЛЬНЕ ВИДАННЯ

РОЗРОБЛЕННЯ МОБІЛЬНОГО ЗАСТОСУНКУ

МЕТОДИЧНІ ВКАЗІВКИ

до лабораторної роботи №2
з дисципліни “Програмування для мобільних платформ”
спеціальності 121 ”Інженерія програмного забезпечення”

Укладач:

Кутельмах Р. К.

Редактор

Комп’ютерне верстання